

NEURAL MACHINE TRANSLATION BY JOINTLY LEARNING TO ALIGN AND TRANSLATE

20220798 정석환

목 차

- 01 연구 배경
- 02 모델 개요
- 03 모델 구조
- 04 Baseline 구현

- 05 Attention Heatmap
- 06 실험 결과
- 07 결론 및 향후 연구 방향

연구 배경

RNN 기반 Seq2Seq 기존 문제점

- 입력 문장을 하나의 고정 길이 벡터로 압축
 - 문장이 길어질수록 정보 손실 발생
 - 장기 의존성 학습 어려움
 - 긴 문장 번역 시 성능 급락

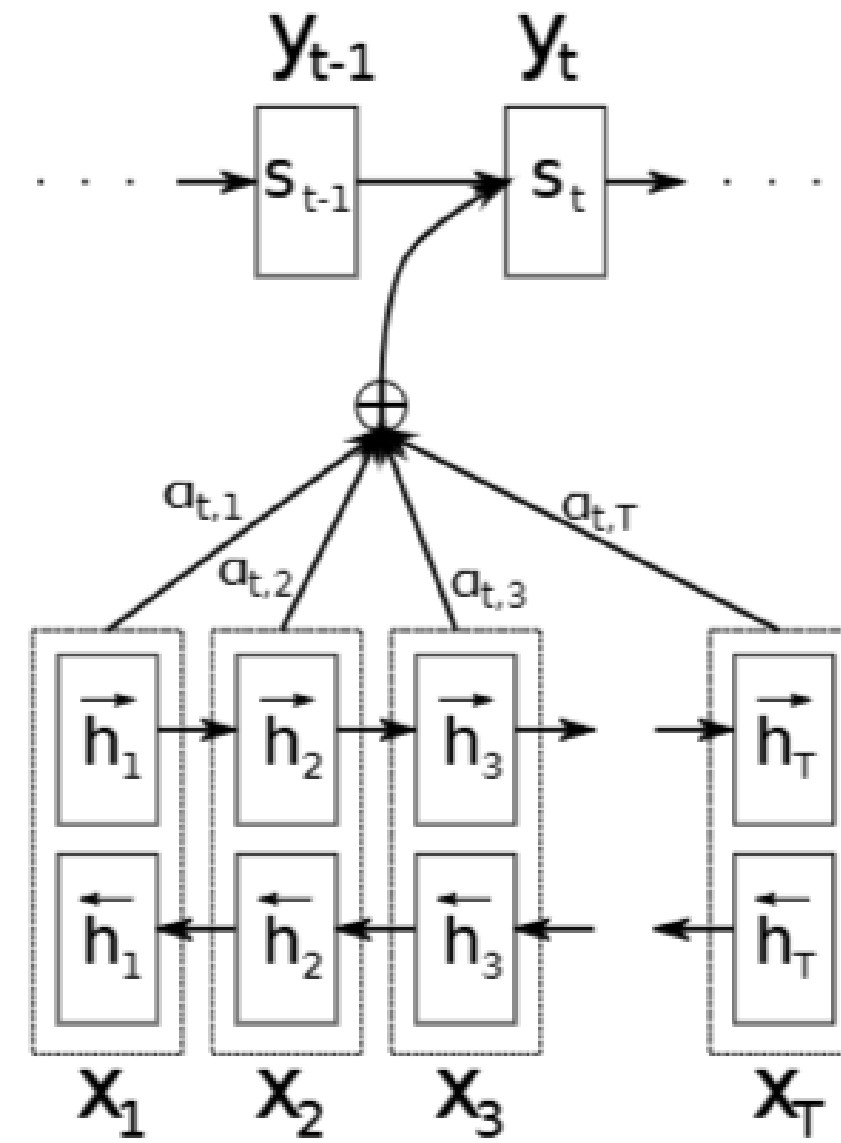
해결 아이디어

- 사람처럼 필요한 부분만 집중해서 보고 번역하자
 - 필요한 위치의 정보를 선택적으로 참조하는 attention이 도입

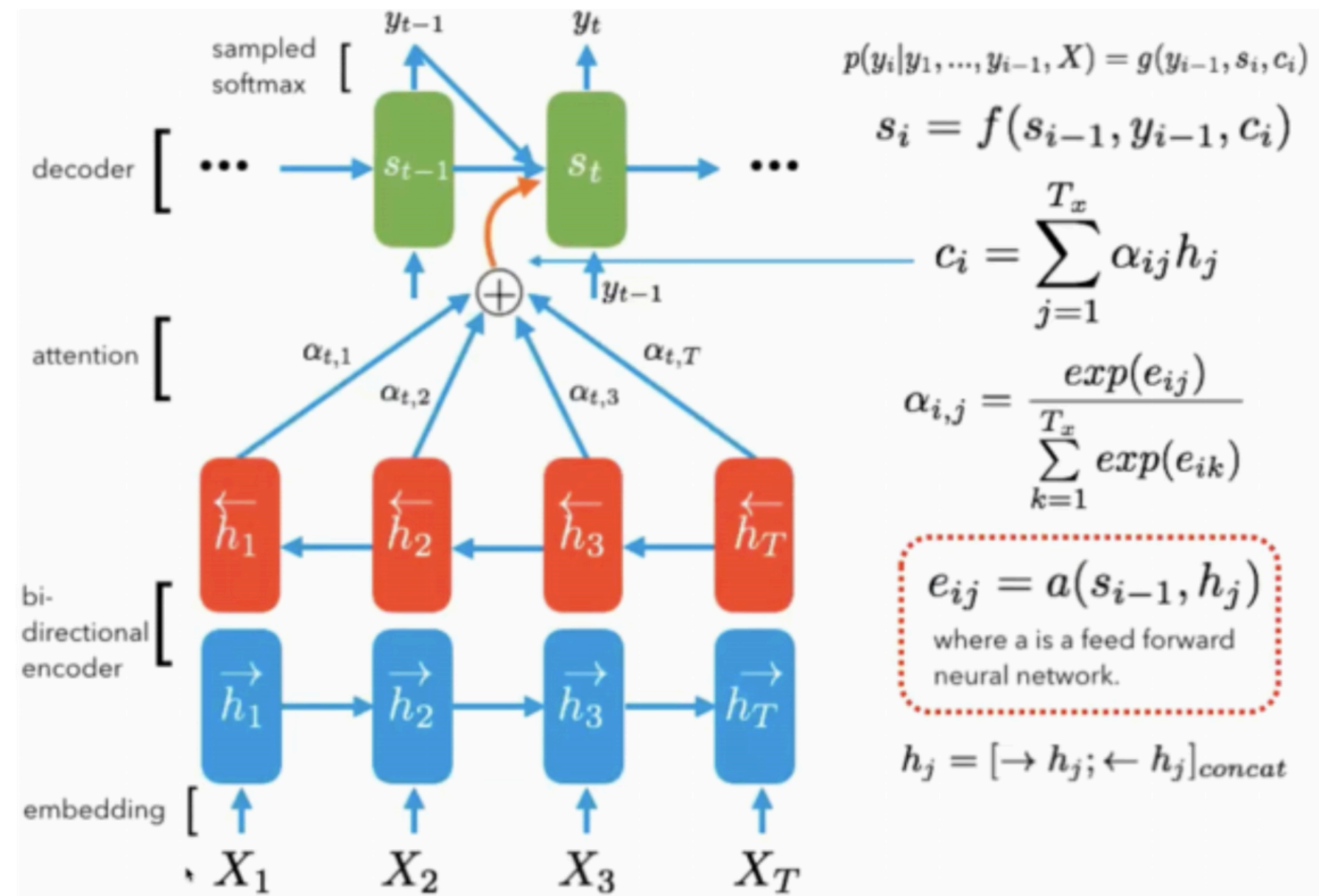
Bahdanau Attention 모델 개요

구조

Encoder의 Bi-RNN hidden state들과
Decoder hidden state를
Additive Attention으로 결합해
context vector를 생성하는 구조



Encoder-Decoder 구조



- Encoder는 양방향 RNN으로 입력 문장을 vectors로 변환
- Decoder는 context vector와 이전 상태를 이용해 다음 단어를 생성

Step 1: Attention Score 계산

Additive Attention Score 계산

- 디코더의 이전 hidden state(s_{t-1}) 사용
- 인코더 hidden state(h_i)와 각각 선형 변환 후 더함
- tanh를 적용한 뒤 W^T 를 곱해 score 계산
- 이러한 과정 때문에 Additive Attention이라 부름

$$score(s_{t-1}, h_i) = W_a^T \tanh(W_b s_{t-1} + W_c h_i)$$

Step 1: Attention Score 계산

1

$$\begin{bmatrix} & & & \\ & & & \\ & & & \\ & & & \end{bmatrix} \times \begin{matrix} h_1 & h_2 & h_3 & h_4 \\ \begin{bmatrix} & & & \\ & & & \\ & & & \\ & & & \end{bmatrix} \end{matrix} = \begin{matrix} W_b s_{t-1} \\ \begin{bmatrix} & & & \\ & & & \\ & & & \\ & & & \end{bmatrix} \end{matrix}$$

$$\begin{bmatrix} & & & \\ & & & \\ & & & \\ & & & \end{bmatrix} \times \begin{matrix} s_{t-1} \\ \begin{bmatrix} \\ \\ \\ \end{bmatrix} \end{matrix} = \begin{matrix} W_c H \\ \begin{bmatrix} \\ \\ \\ \end{bmatrix} \end{matrix}$$

2

$$\tanh \left(\begin{matrix} s_{t-1} \\ \begin{bmatrix} \\ \\ \\ \end{bmatrix} \end{matrix} + \begin{matrix} h_1 & h_2 & h_3 & h_4 \\ \begin{bmatrix} & & & \\ & & & \\ & & & \\ & & & \end{bmatrix} \end{matrix} \right) = \begin{bmatrix} & & & \\ & & & \\ & & & \\ & & & \end{bmatrix}$$

$$\tanh(W_b s_{t-1} + W_c H)$$

3

$$\begin{bmatrix} & & & \end{bmatrix} \times \begin{bmatrix} & & & \\ & & & \\ & & & \\ & & & \end{bmatrix} = \begin{matrix} \text{Attention Score} \\ \begin{bmatrix} & & & \end{bmatrix} \\ h_1 & h_2 & h_3 & h_4 \end{matrix}$$

$$e^t = W_a^T \tanh(W_b s_{t-1} + W_c H)$$

Step 2: Softmax로 Attention Weight 계산

$$\text{softmax} \left(\begin{array}{c} \text{Attention Score} \\ \text{[Green Bar]} \\ h_1 \ h_2 \ h_3 \ h_4 \end{array} \right) = \begin{array}{c} \text{Attention} \\ \text{Distribution} \\ \text{[Red Bar]} \end{array}$$

각 score를 softmax를 이용하여
확률 분포로 변환하고 어떤 단어에 집중할지 결정함

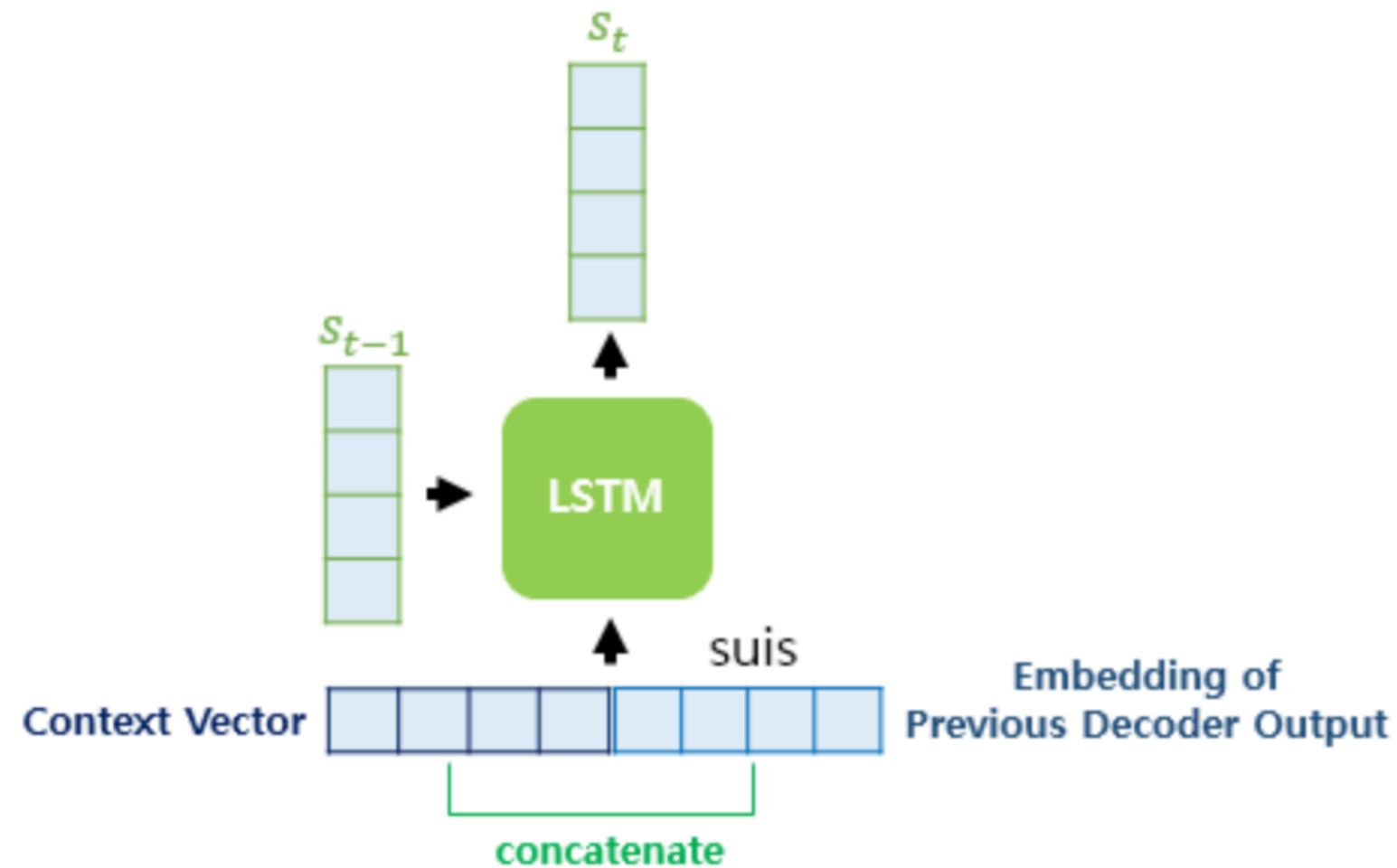
Step 3: Attention Weight로 Context Vector 계산

$$\begin{matrix} h_1 & h_2 & h_3 & h_4 \\ \begin{bmatrix} \square & \square & \square & \square \\ \square & \square & \square & \square \\ \square & \square & \square & \square \\ \square & \square & \square & \square \end{bmatrix} & \times & \begin{bmatrix} \square \\ \square \\ \square \\ \square \end{bmatrix} & = & \begin{bmatrix} \square & \square & \square & \square \end{bmatrix} \end{matrix}$$

Context Vector

각 인코더 은닉 상태에 어텐션 가중치를 곱한 뒤 모두 더해,
입력 문장의 핵심 정보를 담은 context vector를 생성합니다.

Step 4: Context Vector로 Decoder Hidden 계산



이전 디코더 출력 임베딩과 context vector를 concatenate하여 디코더의 입력으로 사용

논문에서 사용한 데이터셋

Model	Updates ($\times 10^5$)	Epochs	Hours	GPU	Train NLL	Dev. NLL
RNNenc-30	8.46	6.4	109	TITAN BLACK	28.1	53.0
RNNenc-50	6.00	4.5	108	Quadro K-6000	44.0	43.6
RNNsearch-30	4.71	3.6	113	TITAN BLACK	26.7	47.2
RNNsearch-50	2.88	2.2	111	Quadro K-6000	40.7	38.1
RNNsearch-50*	6.67	5.0	252	Quadro K-6000	36.7	35.2

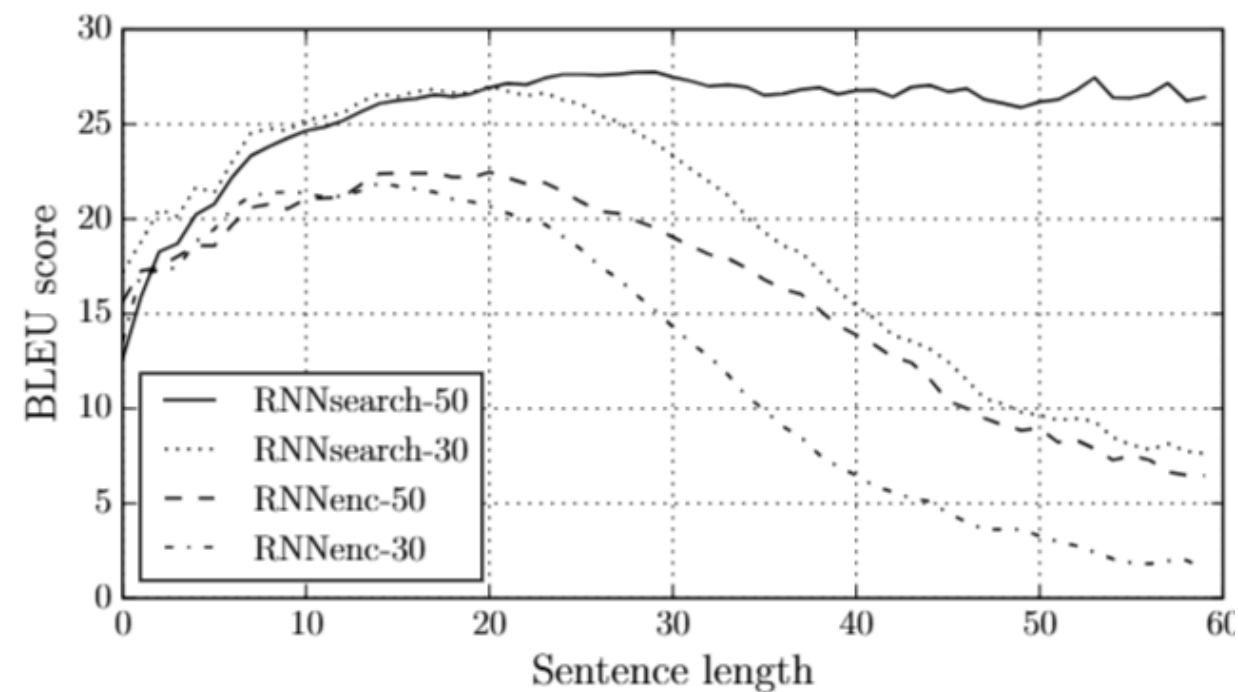
WMT 2014 English-French 데이터 셋

약 1,200만 개의 문장쌍

영어 3.48억 토큰, 프랑스어 3.03억 토큰 규모

어휘는 각 언어별 상위 30,000개의 단어로 제한 나머지는 [UNK] 처리

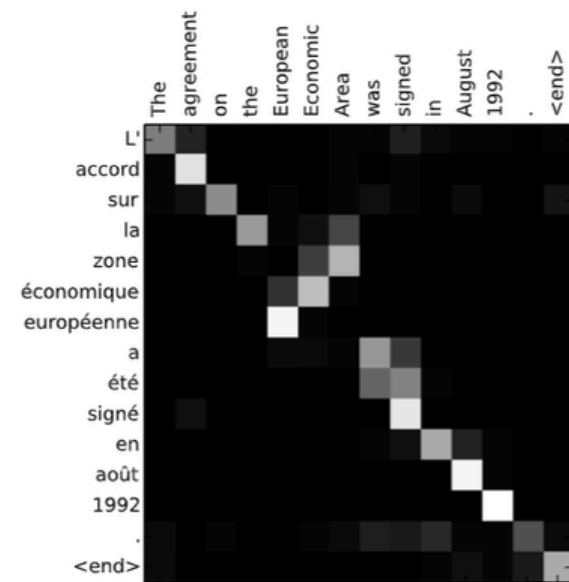
논문 성능



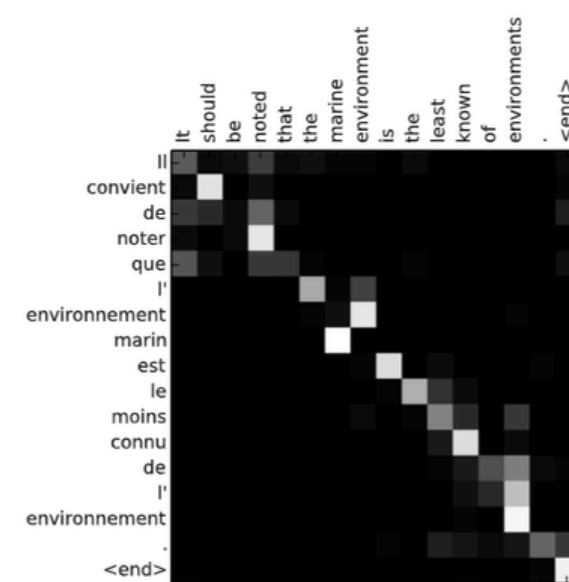
Model	All	No UNK ^o
RNNencdec-30	13.93	24.19
RNNsearch-30	21.50	31.44
RNNencdec-50	17.82	26.71
RNNsearch-50	26.75	34.16
RNNsearch-50 [*]	28.45	36.15
Moses	33.30	35.63

Attention을 적용한 RNNsearch 모델이 기존 RNN Encoder-Decoder보다 BLEU 점수가 5포인트 이상 높았고 긴 문장에서의 성능 저하가 크게 줄어들었습니다.

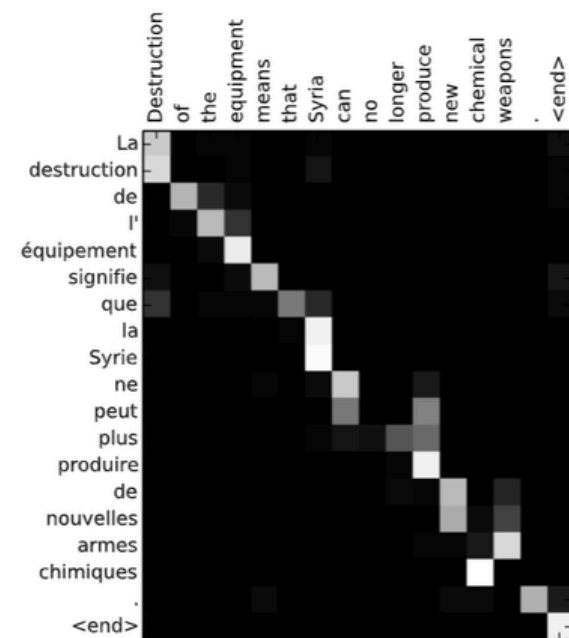
Attention Heatmap



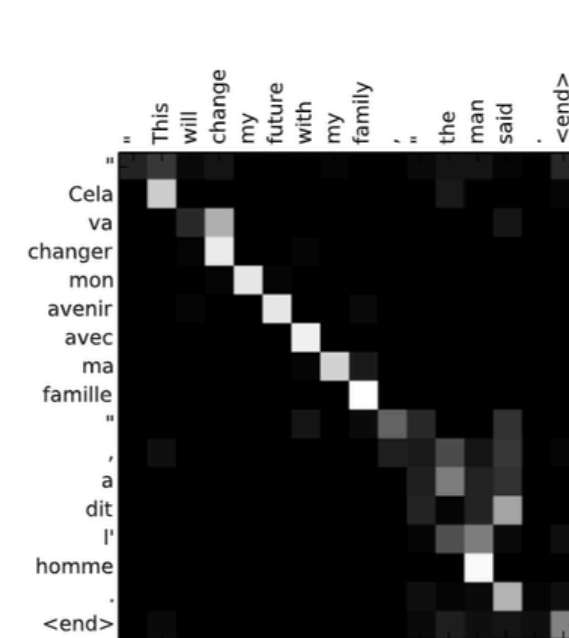
(a)



(b)



(c)



(d)

Attention Heatmap은 입력 문장과
출력 문장 간의 단어 매핑을 보여주며
대각선 형태의 패턴은 번역 과정에서
올바른 단어 정렬이 학습되었음을 의미합니다

BASELINE 학습

```
Batch 430/454 | Loss: 3.0958
Batch 440/454 | Loss: 3.0193
Batch 450/454 | Loss: 3.0744
Epoch 20/20
- Train Loss: 3.0781 | PPL: 21.72
- Val Loss: 3.7675 | PPL: 43.27
- Time Spent: 136.24s
```

BLEU = 29.17, 61.0/36.1/22.6/14.6

학습 결과 요약

- Train Loss: 3.078, Val Loss: 3.767
- Perplexity(PPL): Train 21.72, Val 43.27
- BLEU Score: 29.17
- 총 20 epoch 학습
- Multi30k 데이터 특성상 BLEU가 논문보다 높게 나오는 경향

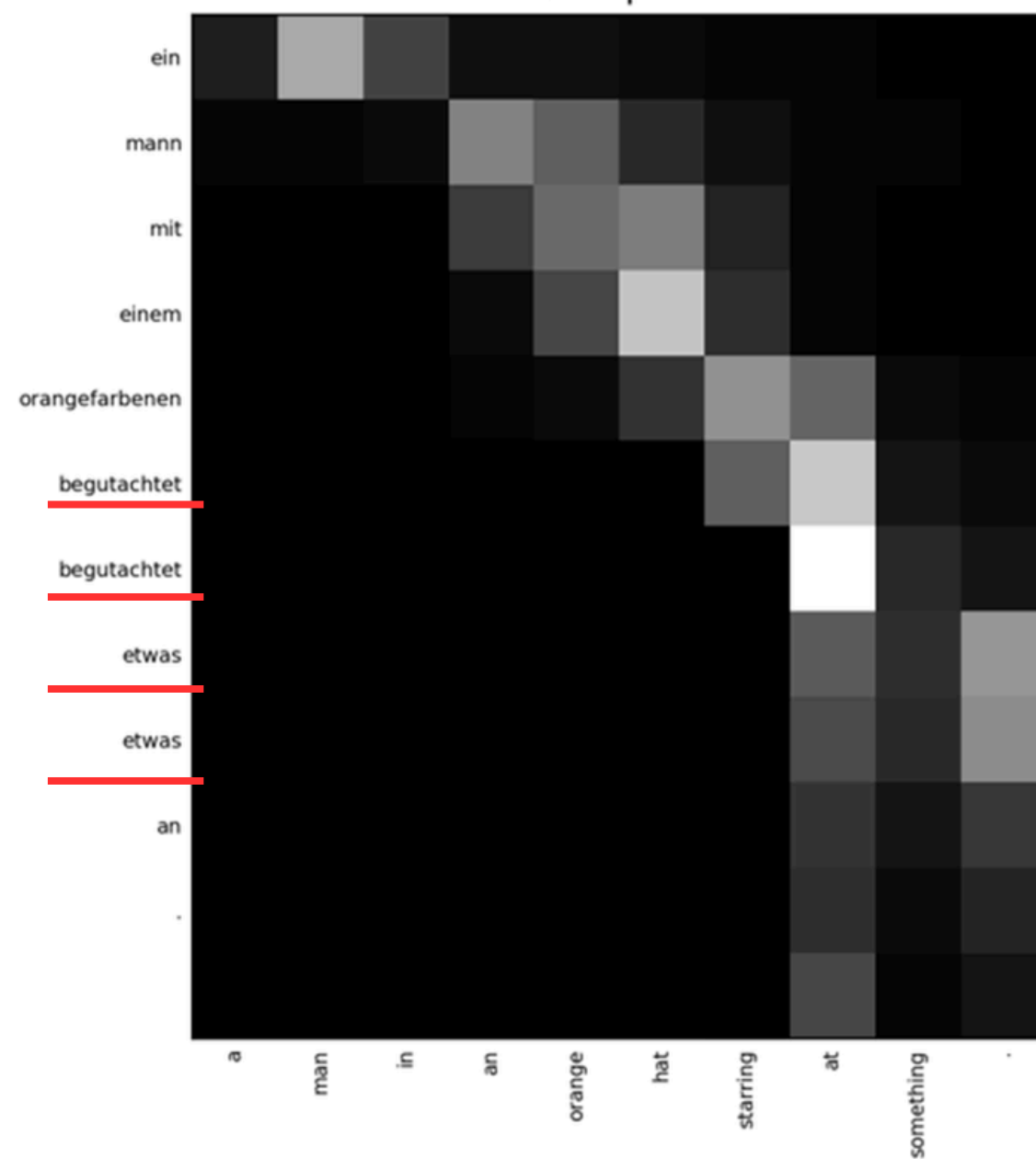
데이터셋 및 파라미터

데이터셋	Multi30k (영-독)
학습 데이터	29,000 문장 쌍
검증 데이터	약 1000 문장 쌍
테스트 데이터	약 1000 문장 쌍
도메인	이미지 캡션
Vocab (English)	5,921 단어
Vocab (German)	7,859 단어
특수 토큰	<pad>, <sos>, <eos>, <unk>
Rare word 처리	1회 등장 단어는 <unk>
GPU	NVIDIA A100 (40GB)

Embedding size	256
Hidden size	512
Optimizer	Adam (lr = 0.0005)
Batch size	64
Epoch	20
Loss Function	CrossEntropyLoss (label smoothing = 0.1)
Teacher Forcing Ratio	1.0 → 0.1
Dropout	Encoder=0.5, Decoder=0.3, Attention=0.1
GPU	NVIDIA A100
학습 시간	약 40분

Attention Heatmap

Example 1



[Source] a man in an orange hat starring at something .
[Prediction] ein mann mit einem orangefarbenen begutachtet begutachtet etwas etwas an .
[Target] ein mann mit einem orangefarbenen hut , der etwas anstarrt .

Attention Heatmap 분석

전체적으로 왼쪽 위 → 오른쪽 아래로 대각선 정렬이 잘 형성됨

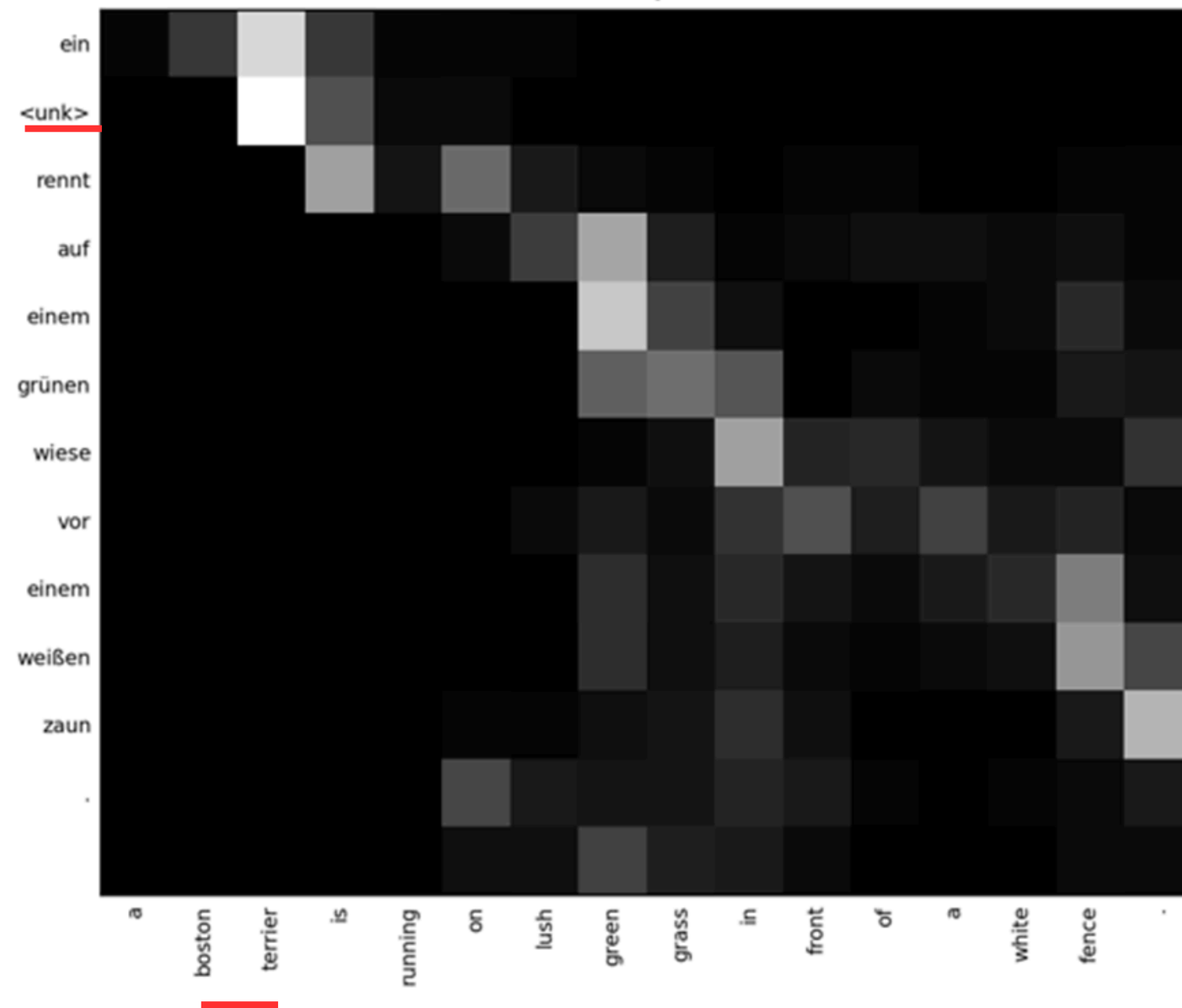
앞부분은 “a man in an orange ...”와 정확하게 대응됨

Heatmap에서 starring at something 에 반복적으로 집중

→ 동일 단어 반복 출력 발생

Attention Heatmap

Example 2



[Source] a boston terrier is running on lush green grass in front of a white fence .
[Prediction] ein <unk> rennt auf einem grünen wiese vor einem weißen zaun .
[Target] ein boston terrier läuft über saftig-grünes gras vor einem weißen zaun .

Attention Heatmap 분석

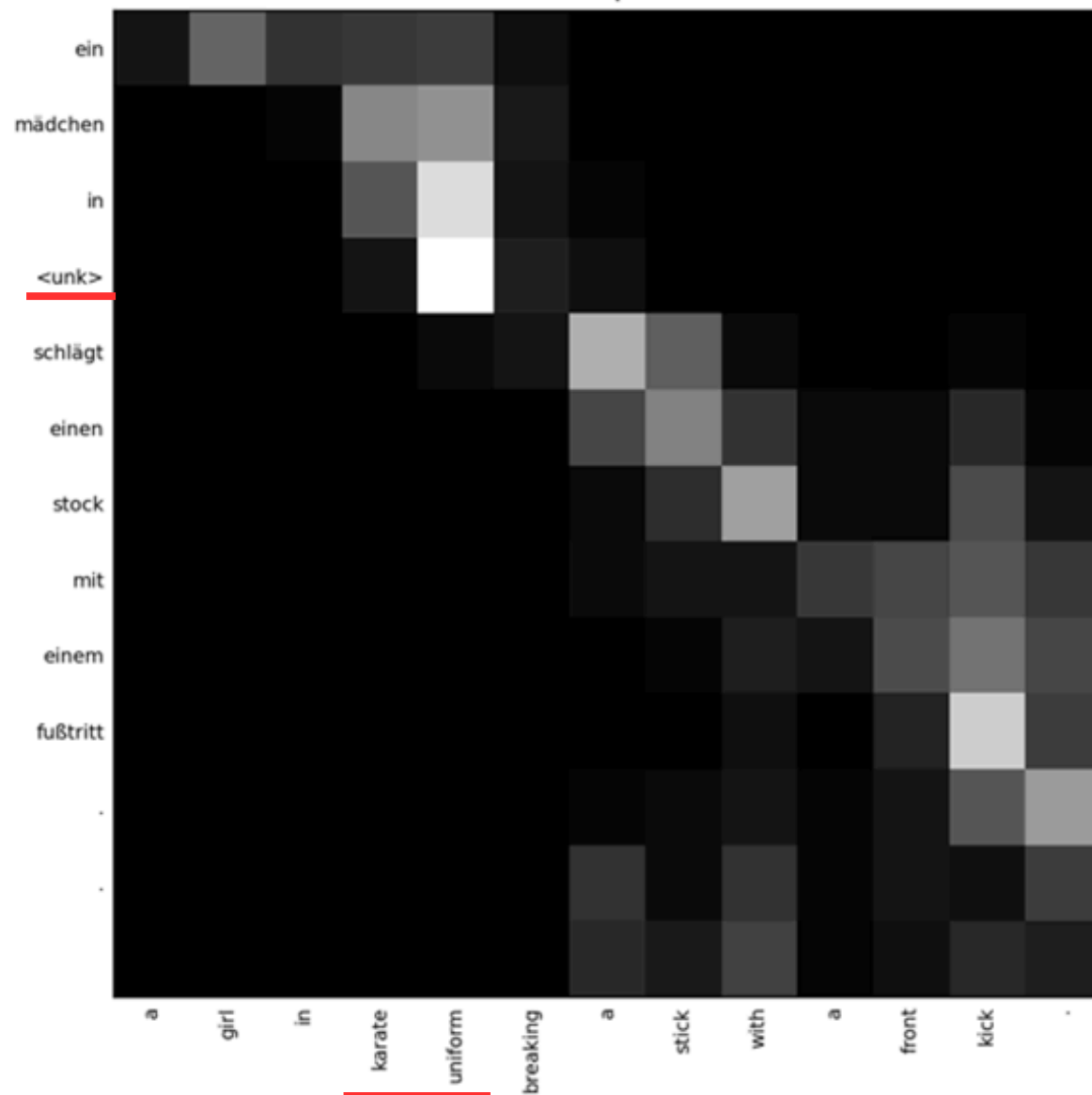
전반적으로 대각선 alignment가 잘 형성되어 단어 대응 자체는 양호

<unk>는 boston / terrier 위치에 강하게 attention

표현은 맞지만 직역 수준의 번역

Attention Heatmap

Example 3



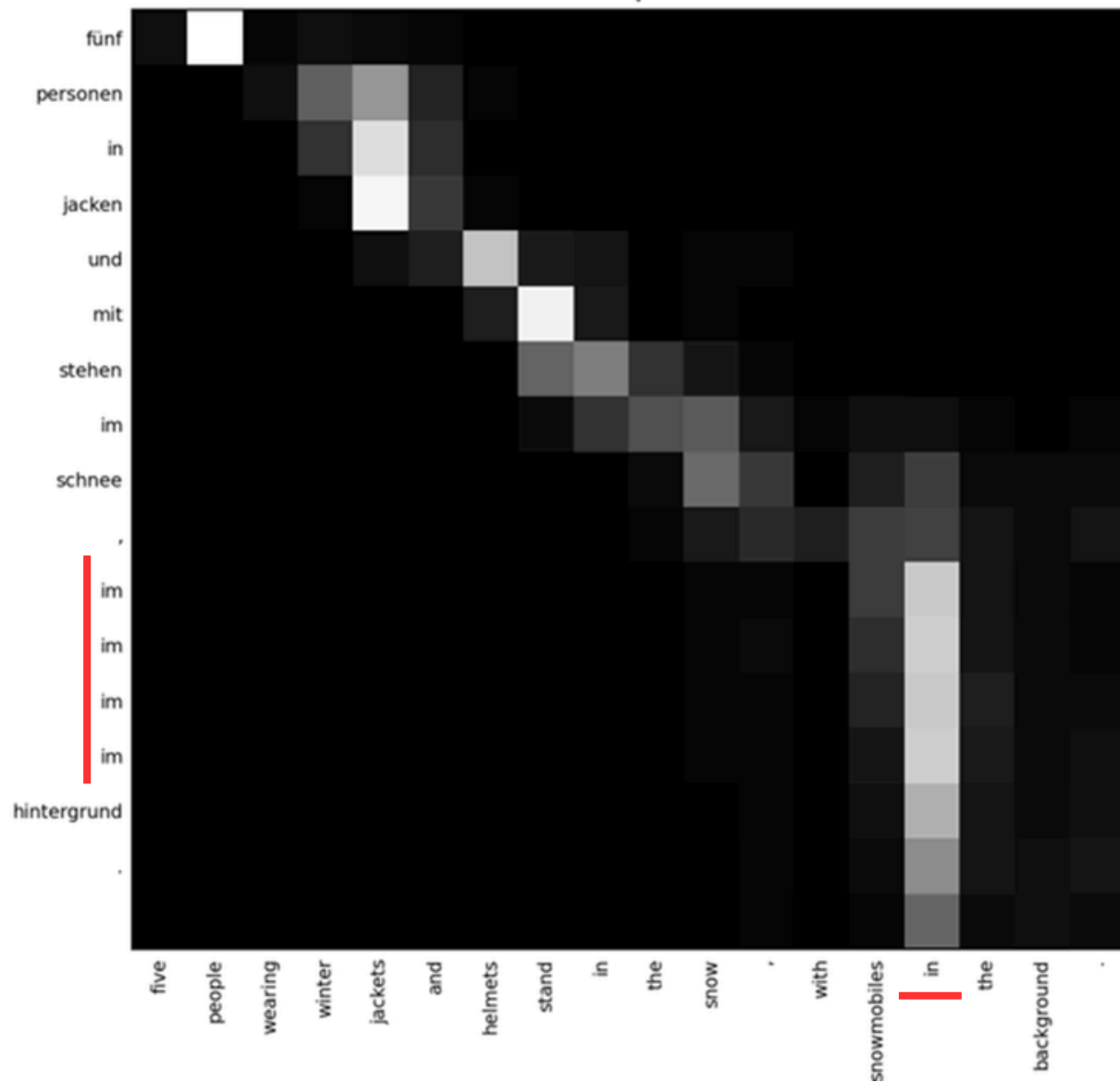
[Source] a girl in karate uniform breaking a stick with a front kick .
[Prediction] ein mädchen in <unk> schlägt einen stock mit einem fußtritt . .
[Target] ein mädchen in einem karateanzug bricht ein brett mit einem tritt .

Attention Heatmap 분석

전반적으로 대각선 alignment가 잘 형성되어 단어 대응 자체는 양호
<unk>는 boston / terrier 위치에 강하게 attention하며 의미는 파악
전반적 의미는 맞지만 표현이 직역 수준으로 자연스러움이 부족함

Attention Heatmap

Example 4



[Source] five people wearing winter jackets and helmets stand in the snow , with snowmobiles in the background .
[Prediction] fünf personen in jacken und mit stehen im schnee , im im im im hintergrund .
[Target] fünf leute in winterjacken und mit helmen stehen im schnee mit schneemobilen im hintergrund .

Attention Heatmap 분석

앞부분 자연스럽게 대각선 alignment 형성

주요 의미(사람 수, 의복, 눈, 배경)는 잘 포착

마지막 문구에서 attention이 특정 위치에 고착되며 반복 출력 발생

→ 동일 단어 반복 출력 발생

결론

- Bahdanau Attention 기반 Seq2Seq 모델을 구현하여 Multi30k 데이터셋에서 성능을 평가하였지만, 데이터 규모 차이로 인해 논문 수준의 번역 품질에는 미치지 못하였고, 반복 생성 등 RNN 기반 모델의 고질적인 한계가 명확하게 드러났다.

향후 연구 방향

- 방안 1) Attention 방식을 바꿔서 alignment 차이를 보는 실험
- 방안 2) RNN 구조의 반복 문제를 근본적으로 해결할 수 있게 self-attention을 사용하는 Transformer로 확장

참고문헌

논문

Bahdanau, D., Cho, K., & Bengio, Y. (2015). Neural Machine Translation by Jointly Learning to Align and Translate. In International Conference on Learning Representations (ICLR 2015)

코드

<https://github.com/Maab-Nimir/Neural-Machine-Translation-by-Jointly-Learning-to-Align-and-Translate>

사진 및 논문 리뷰

<https://velog.io/@xuio/NLP-%EB%85%BC%EB%AC%B8%EB%A6%AC%EB%B7%B0-NEURAL-MACHINE-TRANSLATIONBY-JOINTLY-LEARNING-TO-ALIGN-AND-TRANSLATE>

<https://gbdai.tistory.com/44>

<https://songsite123.tistory.com/75>

<https://wikidocs.net/73161>

<https://wikidocs.net/31695>

<https://meetonfriday.com/posts/4f49bf9b/>

감사합니다